

From Code Repository to Research Infrastructure: Evaluating GitHub for Managing the Scientific Research Lifecycle

Telmo Miguel-Medina^{1*}

¹Electromechanical Engineering Department, University of Burgos,
Burgos, Spain.

Corresponding author(s). E-mail(s): tmiguel@ubu.es;

Abstract

Aims: Research projects generate interdependent artifacts that are usually managed across disconnected tools, with weak traceability of the process that produced the outputs. We ask to what extent GitHub’s native functionalities can serve as an integrated infrastructure for managing scientific research projects across the research lifecycle.

Methods: A hybrid design. (i) A bibliometric analysis of a 5,139-work OpenAlex and arXiv corpus (2008–2025) on digital infrastructure for research-process management, mapped onto lifecycle stages. (ii) A structured review of 67 review, synthesis and primary studies from which fifteen research management requirements were extracted. (iii) A reproducible assessment of GitHub’s 68 native capabilities against those requirements using a four-level rubric. (iv) A reference architecture and a reusable project template synthesised from the mapping. (v) A self-referential implementation of the framework on this study’s own repository, evaluated on six dimensions.

Results: The literature is output-biased: data management and analysis account for 50.8% and 41.9% of the corpus, against 3.7% for idea and question formulation. GitHub supports 13 of 15 requirements at partial or better and 7 directly (mean 2.43 of 3 for the fourteen core requirements), with none unsupported, but support is direct where research resembles software engineering and partial or limited for research-specific traceability of questions, decisions and provenance — the same three requirements the literature under-serves (mean 1.67 versus 2.50). Five requirements need external infrastructure. The reference architecture (15 components, 4 mandatory conventions) and a 33-file template operationalise the covered capabilities. In the self-referential case, the documentation, decision-log and version-control layer worked with near-zero overhead (overall evaluation

1.62 of 2; documentation and coverage at 2.0; usability lowest at 1.25), while the coordination layer was not exercised.

Conclusion: GitHub can serve as an integrated coordination and traceability *layer* for a substantial part of the research lifecycle — strong from execution to output, weak upstream, and dependent on complementary infrastructure for preservation and large data. It maps the research-process-management gap rather than closing it.

1 Introduction

1.1 Complexity and artifact proliferation in modern research

A contemporary research project is a chain of interdependent objects: a research question, a protocol, a literature corpus, data, analysis code, a computational environment, intermediate results, figures, tables, a manuscript, a publication, released software and supplementary material. These objects evolve continuously and are produced by distributed, often multidisciplinary teams. Computational and data-intensive research has amplified both the number of artifacts and the need to keep them consistent with one another [1–3].

1.2 Fragmentation of research-process management

In common practice these artifacts and the activities that generate them are spread across disconnected systems: email, cloud storage, local folders, spreadsheets, reference managers, statistical software, code repositories, task managers and manuscript editors. Each tool serves its local function, but the coordination layer between them is largely absent. Reviews of research data management, scholarly infrastructure and post-award reporting repeatedly associate this fragmentation with weak decision traceability, incomplete process documentation, inconsistent versioning, limited provenance and low project visibility [4–7].

1.3 Managing the process, not only archiving the outputs

Two decades of work on scientific workflow systems, provenance, research data management, virtual research environments and open-science platforms have produced mature evidence on *output* stewardship — repositories, persistent identifiers, curation, dissemination [8–12]. The management of the research *process* that produces those outputs — planning, question formulation, decision tracking, cross-stage coordination — has received comparatively little systematic, quantitative attention [13–15].

1.4 Code-hosting platforms beyond software

GitHub already provides primitives for coordination, versioning, issue tracking, structured discussion, review and automation. Guidance and case studies describe its use for versioned, collaborative and reproducible research in computational biology [16–18], ecology and evolution [19], laboratory research [20], and for developing community

data standards [21]; its footprint in the scholarly record is itself growing and measurable [22]. Yet this evidence is case-based, and studies that mine GitHub as research data caution that its use is heterogeneous and often shallow [23, 24]. Whether GitHub’s primitives can be organised into an infrastructure for managing the research process, and which lifecycle stages such an infrastructure can actually cover, has not been established on an evidentiary basis.

1.5 Aim and research questions

The aim of this study is to characterise the literature on digital infrastructure for research-process management, derive a lifecycle-structured set of research management requirements, determine quantitatively how far GitHub’s native functionalities cover them, consolidate the covered functionalities into a reusable reference architecture and template, and probe feasibility through a self-referential implementation. The study does not claim that GitHub replaces specialised research infrastructure.

RQ1 How has the literature on digital infrastructure for managing the research process evolved in volume, disciplines, venues and thematic structure, and which lifecycle stages are under-represented?

RQ2 What research project management requirements across the research lifecycle can be derived from this literature?

RQ3 To what extent, with what support level and limitations, do GitHub’s native functionalities cover these requirements, and how is coverage distributed across lifecycle stages?

RQ4 How can the covered functionalities be organised into a coherent reusable reference architecture and project template?

RQ5 What does a self-referential implementation, together with the reusable template, reveal about feasibility, traceability gains and limitations relative to a fragmented workflow?

2 Materials and Methods

2.1 Overall design

The study is a multi-method evidence synthesis with three coupled components: a bibliometric analysis of the field (RQ1); a structured requirement extraction and a reproducible requirement–functionality coverage analysis (RQ2, RQ3); and a design synthesis with a single self-referential feasibility test (RQ4, RQ5). All datasets, coding sheets, mapping tables and analysis scripts are openly available (Section 5); every table and figure in this paper is regenerated by a versioned script from those inputs.

2.2 Bibliometric corpus and analysis (RQ1)

The primary corpus was built from OpenAlex, which indexes journal articles, conference papers and preprints with abstracts, concepts and citations and is fully reproducible from a published query. Fifteen `title.and.abstract` queries combining a research-context term with a process-management or infrastructure term (for

example “*research workflow*”, “*research data management*”, “*version control*” + *research*) were run with the filters 2008--2025 and document types article, review and proceedings-article. Five narrower arXiv queries supplemented the version-control and research-software strand. Records were de-duplicated on DOI then on normalised title and year, and data/software-repository deposits (Zenodo, Figshare and similar) were removed, yielding a corpus of **5,139 works**. Scopus and Web of Science were not used; the coverage and precision trade-off is a limitation (Section 4.6), and a heuristic precision estimate for the broadest queries was ≈ 0.94 .

Analysis covered annual production, source and disciplinary profile, country distribution, and a keyword co-occurrence map. Corpus concepts and abstracts were then matched against an eleven-item lexicon of research-lifecycle stages (idea, question, planning, literature, methods, data, analysis, results, dissemination, outputs, governance) to produce a stage-hit profile. Because indexing of 2025 was incomplete at retrieval, growth statements use a 2008–2024 series.

2.3 Structured review and requirement extraction (RQ2)

The requirement extraction rests on a structured evidence base of **52 review, survey and synthesis papers** identified through a deep-search synthesis prioritising secondary literature, complemented by **15 purposively selected primary studies** of Git and GitHub use in research. From this base, literature challenges were traced to management needs and then to derived requirements following the chain *finding* → *problem* → *need* → *requirement* → *category*. Seventeen requirement rows were extracted and consolidated, by thematic grouping and reconciliation with a provisional domain list, into a framework of fifteen requirements (RM1–RM15), each with a definition, expected capabilities, lifecycle stages, evidence, a category, and a relative importance. Requirements rated of high importance but weak in literature attention were flagged as *differentiators*. Extraction was performed by a single coder; the absence of a second coder is a limitation.

2.4 GitHub capability catalogue and mapping rubric (RQ3)

GitHub’s *native* functionality was catalogued as **68 capabilities in twelve feature groups** (repositories and Git, Markdown and documentation, Issues, Projects, Milestones, Labels, Discussions, Branches, Pull Requests, Actions, Releases and tags, and access/identity). Marketplace and third-party applications were excluded; the one retained non-native item is the GitHub–Zenodo release webhook, the de facto standard bridge for persistent identification. Each capability records its function, research use, the durable record it leaves, its plan availability (verified against GitHub documentation) and its limitations.

Each requirement RM1–RM15 was then assigned the best support level GitHub’s native functionality can reach with a reasonable implementation pattern, using a four-level rubric: *Direct* (3) — a native feature meets the requirement; *Partial* (2) — met with an additional process or convention on native features; *Limited* (1) — marginal support only; *Not supported* (0). Every assignment carries an evidence note,

the primary capabilities, an implementation pattern, and any external-tool dependency. A stricter variant capping convention-heavy support at Limited was scored as a robustness check.

2.5 Coverage indicators (RQ3)

From the mapping we computed the support distribution, the mean support overall and by requirement category, the mean for the differentiators versus the rest, the count of requirements needing external tools, and a lifecycle-coverage profile (support weighted by each requirement’s applicability to each stage). Because one requirement (governance and sustainability) is only partly in scope, the headline mean is reported for the fourteen core requirements as well as for all fifteen.

2.6 Reference architecture and template (RQ4)

The Direct- and Partial-support capabilities were synthesised into a layered reference architecture of named components, each a single GitHub configuration serving a set of requirements, together with the conventions that make generic features research-usable, a lifecycle model and seven workflow definitions. The architecture was operationalised as a discipline-independent GitHub template repository, with a manifest mapping every file to the components and requirements it serves.

2.7 Self-referential implementation and evaluation (RQ5)

Following a self-referential design, the framework was applied to the development of this study; the repository is the evidence. Because the work was carried out as a single-author effort, the documentation, decision-log and version-control components were exercised natively while the Issue, Project and Pull Request coordination layer was configured but not used; the activity that would have populated those objects is recorded in a reconstructed register. Each architecture component was classified by implementation status. The framework was then evaluated on six dimensions — requirement coverage, traceability, documentation, organization, transparency, usability — with 21 sub-scores on a 0/1/2 scale, each tagged as observed, retrospective, design-support or planned. A conventional fragmented workflow was contrasted with the framework across six traceability and visibility dimensions.

3 Results

3.1 Bibliometric map of the field (RQ1)

The literature is an assembly of partially connected strands — scientific workflow systems, research data management, provenance, research information systems, open science and version control — rather than one field. In the 5,139-work corpus, annual output rises nearly tenfold between 2008 and 2024. Publications concentrate in computer science, library and information science and the biomedical literature; the leading venues after removing repository deposits are the *International Journal of Digital Curation*, *Concurrency and Computation*, *Future Generation Computer Systems*,

F1000Research and *PLoS ONE*, and the leading countries are the United States, Germany, the United Kingdom, China and the Netherlands. The keyword co-occurrence map (Figure 1) shows the corpus spanning computer science, data science, knowledge management and the social sciences, with workflow, provenance and research data management as recurring bridging terms.

The stage-hit profile is markedly uneven (Table 1). Data management and analysis dominate (50.8% and 41.9% of the corpus); dissemination and coordination follow (24.4% and 21.1%); and idea and question formulation are at the bottom (3.7%), with provenance at 7.0%. A stage-level reading rates seven of fifteen lifecycle stages as thinly covered: idea, question, planning, ethics review, post-publication traceability, cross-stage coordination and governance. The structured review of 52 secondary studies reproduces the same bias independently.

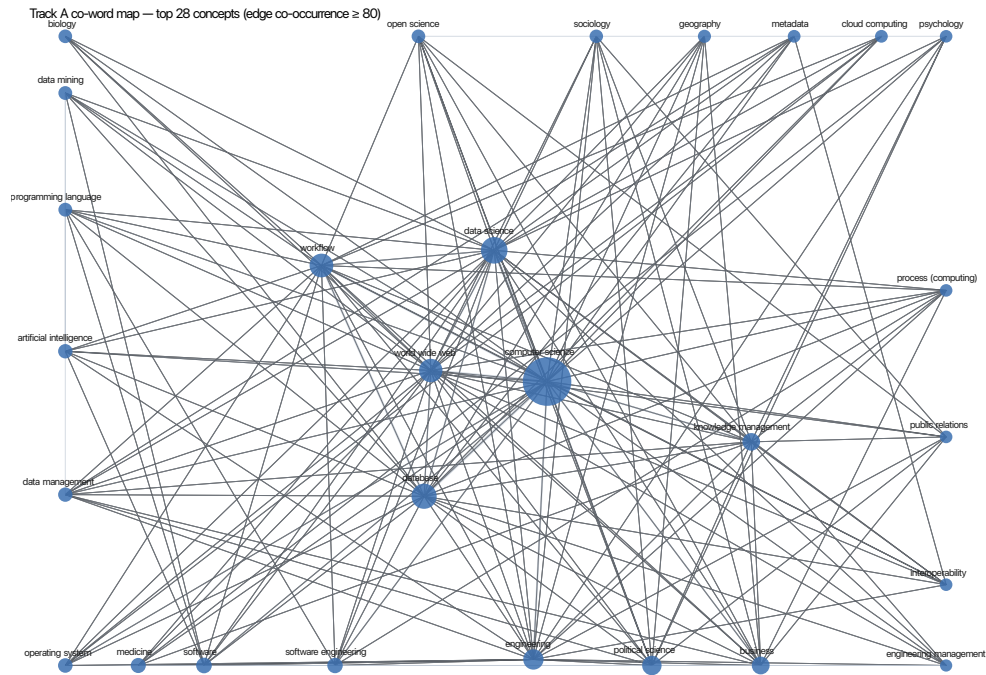


Fig. 1 Keyword co-occurrence map of the 5,139-work corpus (top concepts; edges are strong co-occurrences). The field spans computer science, data science, information management and the social sciences, bridged by workflow, provenance and research data management.

Table 1 Research-lifecycle stage coverage in the bibliometric corpus (RQ1). Share of the 5,139-work corpus whose concepts or abstract match the stage lexicon.

Stage	% of corpus	Stage	% of corpus
Data management	50.8	Planning	12.7
Analysis / workflow	41.9	Literature	10.0
Dissemination	24.4	Outputs / identification	11.9
Coordination	21.1	Governance	10.9
Methods / protocol	13.7	Provenance	7.0
		Idea / question	3.7

3.2 Requirements and GitHub coverage (RQ2, RQ3)

Requirements (RQ2).

Fifteen requirements were derived (Table 2), concentrated in traceability (four), documentation (three) and collaboration (two). Three requirements — research planning (RM1), research-question management (RM2) and decision traceability (RM5) — are of high importance yet weakly attended to in the literature; these differentiators define the gap the study targets. The requirements are interdependent: provenance (RM10) consumes RM2, RM4, RM5, RM8 and RM9, and transparency (RM11) is an emergent property of RM4, RM5, RM8 and RM10.

Coverage (RQ3).

GitHub supports **13 of 15 requirements at Partial or better and 7 at Direct**, with none unsupported. The mean support is **2.43 of 3** for the fourteen core requirements (2.33 for all fifteen). Support is Direct where research work resembles software engineering — task management, review-based collaboration, communication, version control, documentation, transparency and automation, all with category means of 3.0 — and Partial or Limited for research-specific traceability (RM2 Limited; RM5 and RM10 Partial), planning (RM1 Partial) and governance (RM15 Limited) (Table 3). The differentiator requirements have a mean support of **1.67 against 2.50** for the rest: the upstream gap seen bibliometrically in RQ1 reappears, independently, in GitHub’s coverage.

Five requirements cannot be met by GitHub alone — version control of large data (RM8), integration across heterogeneous tools (RM9), reproducibility (RM12), persistent identification and preservation (RM14) and governance (RM15) — and require external infrastructure. The lifecycle-coverage profile is comparatively flat (weighted mean support 2.1–2.4 per stage) but lowest at idea (2.12) and question (2.25) and highest at literature, analysis and manuscript, echoing the RQ1 shape. Issues and the Git repository carry most of the framework; Projects, Actions, Discussions, Branches and Releases each contribute to only a few requirements. Under the stricter rubric the overall mean falls to 2.13, but the distribution (7 Direct, 3 Partial, 5 Limited, 0 unsupported) and the qualitative conclusion are unchanged.

Table 2 Research management requirements (RQ2) and their GitHub support level (RQ3). Support: D Direct, P Partial, L Limited. ★ differentiator (high importance, weak literature attention).

ID	Requirement	Category	Support
RM1★	Research planning and roadmap	Planning	P
RM2★	Research question management	Traceability	L
RM3	Task management	Execution	D
RM4	Documentation	Documentation	D
RM5★	Decision traceability	Traceability	P
RM6	Collaboration and contribution tracking	Collaboration	D
RM7	Communication	Collaboration	D
RM8	Version control	Traceability	D
RM9	Research artifact management and integration	Artifact mgmt	P
RM10	Research provenance and artifact linkage	Traceability	P
RM11	Transparency	Documentation	D
RM12	Reproducibility support	Documentation	P
RM13	Automation	Automation	D
RM14	Research output management and identification	Output mgmt	P
RM15	Governance and sustainability	Cross-cutting	L

Table 3 GitHub coverage by requirement category and for the differentiators (RQ3). Mean support on a 0–3 scale.

Grouping	<i>n</i>	Mean support
Execution / Collaboration / Automation	4	3.00
Documentation	3	2.67
Planning / Traceability / Artifact mgmt / Output mgmt	7	2.00
Governance	1	1.00
Differentiators (RM1, RM2, RM5)	3	1.67
All other requirements	12	2.50
Overall (core 14)	14	2.43
Overall (full 15)	15	2.33
Overall, stricter rubric	15	2.13

3.3 Reference architecture, template and feasibility (RQ4, RQ5)

Architecture (RQ4).

The covered capabilities organise into a **15-component, five-layer architecture** (Figure 2) in which GitHub is a coordination and traceability layer, not a container: coordination (research plan, activity tracker, question register, discussion space), record (documentation set, decision log, linkage discipline, change history), production (artifact workspace, review workflow, automation suite), release (release process, archival bridge, governance setup) and a cross-cutting transparency surface. Every requirement is served by at least one component. One component — *linkage discipline*

(mandatory cross-referencing of every commit, pull request and issue) — and four named conventions (a phased plan of record, a question register, a decision-record convention and the linkage discipline itself) are the framework’s contribution over a plain repository: they are what lift RM1, RM2, RM5 and RM10 from raw features toward research-usable support. The end-to-end traceability path they produce is shown in Figure 3. Seven workflow definitions specify the concrete step sequences (registering a research question, executing a task, recording a decision, reviewing an artifact, releasing and archiving, checking reproducibility, onboarding).

Template (RQ4).

The architecture is operationalised as a **33-file discipline-independent template**: a repository skeleton, a documentation set that states the four conventions, three issue forms, a pull-request checklist, a label set, Project field definitions, starter automation and a release checklist. A manifest maps every file to the components and requirements it serves; all fifteen requirements are covered.

Feasibility (RQ5).

In the self-referential implementation, **8 of 15 components were exercised** natively or partially. The record layer (23 documentation files, 19 logged decisions), the artifact workspace and version control were realised at full strength; the coordination layer (Issues, Project, Pull Requests) was configured but not used under single-author conditions. The six-dimension evaluation (Table 4) scores **1.62 of 2 overall** (1.76 for the seventeen observed sub-scores): requirement coverage and documentation reach 2.0, and usability is lowest at 1.25 — driven by configuration complexity, the assumed Git/Markdown literacy, and coordination-layer overhead that a solo effort does not incur or test. Against a fragmented workflow (Table 5), the framework’s benefit is concentrated in linking artifacts that would otherwise stay disconnected — decision rationale, version history and the question-to-result chain become reconstructable — and its cost is concentrated in the coordination layer.

Table 4 Framework evaluation on six dimensions (RQ5), 0–2 scale, from the self-referential case study. “Observed” marks sub-scores evidenced by the actual repository state.

Dimension	Basis	Score
E1 Requirement coverage	design-support	2.00
E2 Traceability	mostly observed	1.50
E3 Documentation	observed	2.00
E4 Organization	observed	1.75
E5 Transparency	observed; external visibility planned	1.50
E6 Usability	observed	1.25
Overall (21 sub-scores)		1.62
Overall, observed only (17)		1.76



Fig. 2 GitHub-based research management reference architecture (RQ4): five layers, fifteen components. GitHub acts as a coordination and traceability layer; large data, computational environments, persistent identifiers and institutional governance remain external and are linked by reference.

4 Discussion

4.1 Principal findings

The study’s central result is a convergence. The gap identified bibliometrically in RQ1 — the literature under-serves idea and question formulation, planning and decision traceability — and the gap measured in RQ3 — GitHub supports those same requirements at 1.67 against 2.50 for the rest — are the same gap, reached by two independent routes. GitHub covers the execution-to-output span of the research lifecycle well and the upstream, research-specific layer poorly; the partiality is systematic, not incidental.

4.2 GitHub as a coordination and traceability layer

Where research work can be expressed as files under version control, discrete tasks, branch-and-review changes and event-triggered automation, GitHub offers mature, purpose-built support, and the self-referential implementation confirms that this layer is low-cost: a full plan of record, nineteen logged decisions and a reproducible analysis pipeline were produced without coordination overhead. Where the object of management is the evolving intellectual process — the questions asked, the choices made and why — GitHub offers only generic features on which a discipline must be imposed. The framework’s contribution is to name that discipline: a mandatory linkage convention and three record conventions that make a plain repository into a traceable one. Even

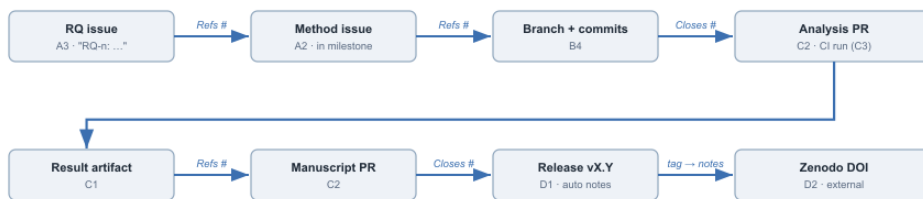


Fig. 3 End-to-end traceability path produced by the linkage-discipline convention: a research-question issue is referenced by the method issue, whose branch and commits feed a reviewed pull request, whose result feeds the manuscript and a tagged release archived with a DOI. Any node is reachable from any other via recorded cross-references.

so, the differentiator requirements remain Partial or Limited; the framework maps the gap rather than closing it.

4.3 Which lifecycle stages benefit

Both the literature and GitHub’s coverage are weakest at the earliest stages. The execution-to-output span — methods, data, analysis, results, manuscript, release — benefits most, because its artifacts are files and its activities are changes. The idea, question and planning stages benefit least, because they are deliberative and their “artifacts” are decisions. This is where tool development and reporting standards — for decision records and research-question provenance — would have the most leverage.

4.4 Practical implications

For an individual researcher or a small group, the practical recommendation is to adopt the record and version-control layer first: a version-controlled `docs` directory, a decision log and disciplined commit messages deliver most of the traceability benefit at little cost. The Issue, Project and Pull Request layer should be added when the project has collaborators, where its coordination value accrues and its overhead is justified. The 33-file template lowers the setup cost of both.

4.5 Relationship with output-preservation workflows

This study addresses the management of the research process; a complementary line of work addresses the preservation and persistent identification of research *outputs*

Table 5 Fragmented workflow versus the GitHub-based framework (RQ5).

Dimension	Fragmented workflow	GitHub-based framework
Task traceability	tasks disconnected from the artifacts they produce	issues plus board plus closing keywords bind task to commit to pull request
Decision traceability	decisions in email and notes, rationale lost	numbered decision log plus decision issue, with rationale and affected artifacts
Version traceability	files renamed “final”; no history	Git line-level history, diff, restore, tags
Documentation	scattered documents; drift; no history	version-controlled Markdown from project start, with per-folder READMEs
Research provenance	no systematic link between question, data and result	cross-references and timelines assemble the chain; release notes inherit it
Project visibility	known only to the email loop	public repository plus status, activity log and changelog

through GitHub, Zenodo and ORCID. The two are the upstream and downstream halves of one lifecycle: the framework’s release and archival-bridge components hand a versioned, cross-referenced record to an external archive that assigns a DOI. GitHub-based process management extends, rather than replaces, the open-science tooling around outputs.

4.6 Limitations

Six limitations bound the findings. First, the framework maps the upstream gap but does not close it: RM1, RM2 and RM5 stay Partial or Limited even with the prescribed conventions. Second, the coordination layer’s collaborative benefit is unproven: the single-author implementation meant that RM6, RM7 and the Issue/Project/Pull Request layer were scored on design support, not observed use. Third, the evaluation is single-case, self-referential, scored by a single coder, on a repository not yet public; the requirement extraction and the support scores likewise had no second rater. Fourth, the bibliometric corpus is built from OpenAlex and arXiv rather than Scopus or Web of Science, trading precision for openness and reproducibility (broad-query precision ≈ 0.94). Fifth, roughly a third of the coverage rests on mandatory convention rather than native features, as the stricter-rubric mean of 2.13 shows. Sixth, GitHub is not a complete research infrastructure: persistent identifiers and preservation, computational environments, large datasets and institutional governance are all external.

4.7 Future research

Three directions follow. A multi-author, multi-project evaluation would test the coordination layer’s collaborative value and the framework’s transferability across disciplines. Instrumenting the linkage-discipline convention — automatically checking

that commits, pull requests and issues cross-reference their research question and decisions — would move provenance from a manual practice toward a verifiable one. And a bibliometric follow-up on the upstream stages specifically would quantify whether the idea/question/planning gap is beginning to close as AI-assisted planning and digital ethics platforms mature.

5 Conclusions

GitHub can serve as an integrated coordination and traceability *layer* for a substantial part of the scientific research lifecycle. It supports thirteen of fifteen literature-derived research management requirements at partial or better and seven directly, with none unsupported; it is strong from execution to output and for research that resembles software engineering; and it is weak for the upstream, research-specific traceability of questions, decisions and provenance — the very requirements the literature also under-serves. Five requirements need complementary external infrastructure. A fifteen-component reference architecture, four mandatory conventions and a thirty-three-file template operationalise what GitHub covers, and a self-referential implementation shows that its documentation and version-control layer works at near-zero overhead while the collaborative value of its coordination layer remains to be demonstrated. GitHub maps the research-process-management gap rather than closing it, and it is best understood not as a replacement for research infrastructure but as the coordination and traceability layer that a fragmented set of research tools currently lacks.

Declarations

Data and code availability

The bibliographic corpus, coding sheets, requirement and mapping tables, reference architecture, template and all analysis scripts are openly available in the study repository at <https://github.com/telmomm/github-research-infrastructure-study> and will be archived with a DOI on Zenodo at release. Every table and figure in this paper is regenerated from those inputs by a versioned script.

Competing interests

The author declares no competing interests.

Funding

This research received no specific grant from any funding agency.

References

- [1] Liew, C.S., Atkinson, M.P., Galea, M., Ang, T.F., Martin, P., Hemert, J.I.: Scientific workflows: Moving across paradigms. *ACM Computing Surveys* **49**, 1–39 (2016) <https://doi.org/10.1145/3012429>

- [2] Liu, J., Pacitti, E., Valduriez, P., Mattoso, M.: A survey of data-intensive scientific workflow management. *Journal of Grid Computing* **13**, 457–493 (2015) <https://doi.org/10.1007/s10723-015-9329-8>
- [3] Mattoso, M., Dias, J., Ocaña, K.A.C.S., Ogasawara, E., Costa, F., Horta, F., Sousa, V., Oliveira, D.: Dynamic steering of HPC scientific workflows: A survey. *Future Generation Computer Systems* **46**, 100–113 (2015) <https://doi.org/10.1016/j.future.2014.11.017>
- [4] Perrier, L., Blondal, E., Ayala, A.P., Dearborn, D., Kenny, T., Lightfoot, D., Reka, R., Thuna, M., Trimble, L., MacDonald, H.: Research data management in academic institutions: A scoping review. *PLoS ONE* **12** (2017) <https://doi.org/10.1371/journal.pone.0178261>
- [5] Donner, E.K.: Research data management systems and the organization of universities and research institutes: A systematic literature review. *Journal of Librarianship and Information Science* **55**, 261–281 (2022) <https://doi.org/10.1177/09610006211070282>
- [6] Crane, K., Blatch-Jones, A., Fackrell, K.: The post-award effort of managing and reporting on funded research: a scoping review. *F1000Research* **12** (2023) <https://doi.org/10.12688/f1000research.133263.2>
- [7] Tutko, A., Henley, A.Z., Mockus, A.: How are software repositories mined? a systematic literature review of workflows, methodologies, reproducibility, and tools. *arXiv abs/2204.08108* (2022) <https://doi.org/10.48550/arxiv.2204.08108>
- [8] Davidson, S.B., Freire, J.: Provenance and scientific workflows: challenges and opportunities. In: *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, pp. 1345–1350 (2008). <https://doi.org/10.1145/1376616.1376772>
- [9] Pérez, B., Rubio, J., Sáenz-Adán, C.: A systematic review of provenance systems. *Knowledge and Information Systems* **57**, 495–543 (2018) <https://doi.org/10.1007/s10115-018-1164-3>
- [10] Ho, R.C.Y., Wong, S.N., Chia, P., Tang, C., Ng, M.: Research data management services in academic libraries to support the research data life cycle: A systematic review. an annual review of information science and technology (ARIST) paper. *Journal of the Association for Information Science and Technology* **77**, 272–300 (2025) <https://doi.org/10.1002/asi.70008>
- [11] Kirkham, J.J., Penfold, N.C., Murphy, F., Boutron, I., Ioannidis, J.P., Polka, J.K., Moher, D.: Systematic examination of preprint platforms for use in the medical and biomedical sciences setting. *BMJ Open* **10** (2020) <https://doi.org/10.1136/bmjopen-2020-041849>

- [12] Mayernik, M.S., Hart, D.L., Maull, K.E., Weber, N.M.: Assessing and tracing the outcomes and impact of research infrastructures. *Journal of the Association for Information Science and Technology* **68** (2017) <https://doi.org/10.1002/asi.23721>
- [13] Cox, A., Tam, W.: A critical analysis of lifecycle models of the research process and research data management. *Aslib Journal of Information Management* **70**, 142–157 (2018) <https://doi.org/10.1108/ajim-11-2017-0251>
- [14] Holler, J., Kedron, P.: Mainstreaming metadata into research workflows to advance reproducibility and open geographic information science. In: *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences* (2022). <https://doi.org/10.5194/isprs-archives-xxviii-4-w1-2022-201-2022>
- [15] Devkota, N., Siddique, M., Karki, D., Thapa, D.: Research in the age of AI — how to plan? a bibliometric exploration of the research lifecycle. *International Research Journal of MMC* (2026) <https://doi.org/10.3126/irjmmc.v7i3.97169>
- [16] Sandve, G.K., Nekrutenko, A., Taylor, J., Hovig, E.: Ten simple rules for reproducible computational research. *PLoS Computational Biology* **9**(10), 1003285 (2013) <https://doi.org/10.1371/journal.pcbi.1003285>
- [17] Blischak, J.D., Davenport, E.R., Wilson, G.: A quick introduction to version control with Git and GitHub. *PLoS Computational Biology* **12**(1), 1004668 (2016) <https://doi.org/10.1371/journal.pcbi.1004668>
- [18] Pérez-Riverol, Y., Gatto, L., Wang, R., Sachsenberg, T., Uszkoreit, J., Veiga Leprevost, F., Fufezan, C., Ternent, T., Eglén, S.J., Katz, D.S., Pollard, T.J., Kononov, A., Flight, R.M., Blin, K., Vizcaíno, J.A.: Ten simple rules for taking advantage of Git and GitHub. *PLoS Computational Biology* **12**(7), 1004947 (2016) <https://doi.org/10.1371/journal.pcbi.1004947>
- [19] Braga, P.H.P., Hébert, K., Hudgins, E.J., Scott, E.R., Edwards, B.P.M., Sylvain, J.D., Beaubien, S., Bouchard, F., Colwill, S., Domke, S., Fisher, J.A.D., Hunter, F.F., Prokopiuk, M.: Not just for programmers: How GitHub can accelerate collaborative and reproducible research in ecology and evolution. *Methods in Ecology and Evolution* **14**(6), 1364–1380 (2023) <https://doi.org/10.1111/2041-210X.14108>
- [20] Chen, K.M., Toro-Moreno, M., Subramaniam, A.R.: GitHub enables collaborative and reproducible laboratory research. *PLoS Biology* **23**(3), 3003029 (2025) <https://doi.org/10.1371/journal.pbio.3003029>
- [21] Crystal-Ornelas, R., Varadharajan, C., Bond-Lamberty, B., Boye, K., Burrus, M., Cholia, S., Crow, M., Damerow, J., Devarakonda, R., Ely, K., Goldman, A., Heinz, S., Hendrix, V., Kakalia, Z., Pennington, S., Robles, E., Rogers, A., Simmonds, M., Velliquette, T., Agarwal, D.: A guide to using GitHub for developing and

- versioning data standards and reporting formats. *Earth and Space Science* **8** (2021) <https://doi.org/10.1029/2021ea001797>
- [22] Escamilla, E., Klein, M., Cooper, T., Rampin, V., Weigle, M.C., Nelson, M.L.: The rise of GitHub in scholarly publications. In: *Linking Theory and Practice of Digital Libraries (TPDL 2022)*. Lecture Notes in Computer Science, vol. 13541, pp. 187–200 (2022). https://doi.org/10.1007/978-3-031-16802-4_15
- [23] Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D.M., Damian, D.: An in-depth study of the promises and perils of mining GitHub. *Empirical Software Engineering* **21**, 2035–2071 (2016) <https://doi.org/10.1007/s10664-015-9393-5>
- [24] Trisovic, A., Lau, M.K., Pasquier, T., Crosas, M.: A large-scale study on research code quality and execution. *Scientific Data* **9**, 60 (2022) <https://doi.org/10.1038/s41597-022-01143-6>